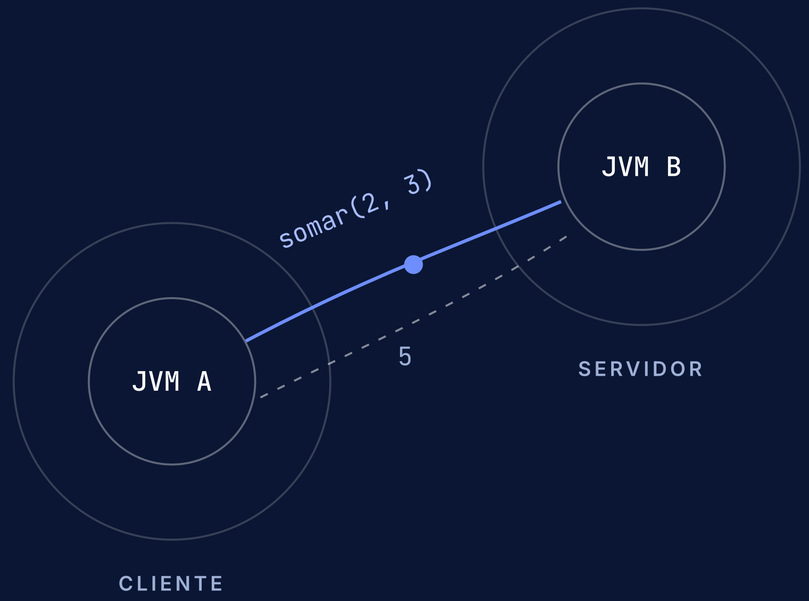
JAVA · JAVA.RMI

RMI

Remote Method Invocation

Invocação remota de métodos em Java



INTEGRANTES

Carlos Eduardo da Silva Pechiori

Gustavo Wendel Lopes da Silva

Rhuan Leonardo Baffero Marques

Sara Batista Pereira

[1] ORACLE. Java Remote Method Invocation Specification. Java SE 25. [2] WOLLRATH, A.; RIGGS, R.; WALDO, J. A distributed object model for the Java system. Computing Systems, v. 9, n. 4, 1996.

O QUE VAMOS VER

Roteiro

Seis paradas, quatro vozes e uma analogia de restaurante do começo ao fim.

≈ 70 minutos + perguntas

Nº	TEMA	QUEM
00	Revisão relâmpago de POO	Apresentador 1
01	Introdução à invocação remota de métodos	Apresentador 1
02	Conceitos básicos	Apresentador 2
03	Camadas do RMI	Apresentador 3
04	Serviços de registro e naming	Apresentador 3
05	Aplicações de RMI — e o que mudou até hoje	Apresentador 4

Tópicos definidos pela disciplina. As fontes de cada conteúdo estão no rodapé do respectivo slide e reunidas na página de Referências.

E se o objeto que você quer usar estiver em *outro computador*?

Até aqui, nossos objetos sempre viveram juntos, dentro de um mesmo programa. O RMI foi criado justamente para quebrar essa regra.[2,3]

[2] WOLLRATH, A.; RIGGS, R.; WALDO, J. A distributed object model for the Java system. Computing Systems, v. 9, n. 4, 1996.

[3] ORACLE. An Overview of RMI Applications. The Java Tutorials.

00

APRESENTADOR 1

Revisão relâmpago de POO

Quatro ideias que o RMI usa o tempo todo:
objetos, interfaces, exceções e serialização.

[4] ORACLE. Lesson: Object-Oriented Programming Concepts. The Java Tutorials.

REVISÃO · 1 DE 3

Um objeto é alguém a quem você *pede* coisas

C **Classe**

A planta da casa: descreve como os objetos serão.[4]

O **Objeto**

A casa construída: tem estado (dados) e comportamento.[4]

M **Método**

O que o objeto sabe fazer. Chamar um método = fazer um pedido.[4]

```
ContaBancaria conta = new ContaBancaria();

conta.depositar(100);
double saldo = conta.getSaldo();
```

No RMI, a sintaxe é a mesma. A diferença: o objeto `conta` pode estar **em outro computador**.[3]

[3] ORACLE. An Overview of RMI Applications. The Java Tutorials. [4] ORACLE. Lesson: Object-Oriented Programming Concepts. The Java Tutorials.

REVISÃO · 2 DE 3

Interface é um *contrato*

Como um cardápio: diz **o que** pode ser pedido, não **como** a cozinha prepara.[5]

Polimorfismo: quem usa a interface não precisa saber qual classe está “do outro lado” — só que ela cumpre o contrato.[6]

CONTRATO

```
public interface Calculadora {
    int somar(int a, int b);
}

class CalculadoraSimples implements Calculadora {
    public int somar(int a, int b) {
        return a + b;
    }
}

Calculadora c = new CalculadoraSimples();
c.somar(2, 3);   // 5
```

→ É por isso que o RMI consegue entregar ao cliente um **substituto** que “finge” ser o objeto real: o cliente só enxerga a interface.[2]

[2] WOLLRATH, A.; RIGGS, R.; WALDO, J. A distributed object model for the Java system. Computing Systems, v. 9, n. 4, 1996. **[5]** ORACLE. What Is an Interface? The Java Tutorials. **[6]** ORACLE. Polymorphism. The Java Tutorials.

REVISÃO · 3 DE 3

Duas ferramentas que vão aparecer muito

Exceções

O plano B

`throws` avisa que algo pode dar errado; `try/catch` decide o que fazer. Exceções **checadas** são obrigatórias: o compilador exige que você trate.[7]

```
try {
    arquivo.ler();
} catch (IOException e) {
    System.out.println("Deu ruim: " + e);
}
```

Serialização

Objeto que vira bytes

Transforma um objeto em uma sequência de bytes para **salvar ou enviar** — e depois remonta do outro lado. Como desmontar um móvel para caber na caixa da mudança.[8,9]

```
class Pedido implements Serializable {
    String prato;
    int quantidade;
}
```

[7] ORACLE. Lesson: Exceptions. The Java Tutorials. [8] ORACLE. Java Object Serialization Specification. Java SE 25. [9] ORACLE. Object Streams. The Java Tutorials.

01

APRESENTADOR 1

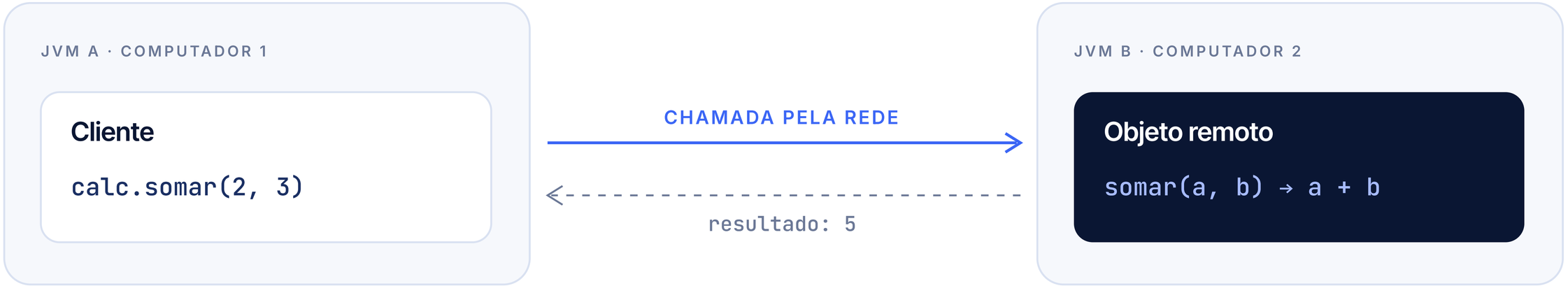
Introdução à invocação remota de métodos

O que é, de onde veio e por que **“remoto”** nunca é igual a **“local”**.

[3] ORACLE. An Overview of RMI Applications. The Java Tutorials.

RMI EM UMA FRASE

Uma API do Java que permite a um objeto em uma JVM chamar métodos de um objeto que está *em outra JVM* — muitas vezes, em outro computador.[3,1]



JVM (Java Virtual Machine) é o "motor" que executa um programa Java. Dois programas rodando = duas JVMs, cada uma com sua própria memória.[10]

[1] ORACLE. Java Remote Method Invocation Specification. Java SE 25. **[3]** ORACLE. An Overview of RMI Applications. The Java Tutorials. **[10]** LINDHOLM, T. et al. The Java Virtual Machine Specification: Java SE 25 Edition. Oracle, 2025.

A ANALOGIA QUE VAI NOS ACOMPANHAR

RMI é como pedir num *restaurante*

Analogia didática do grupo, apoiada na ideia de **proxy**: um representante local do objeto remoto.[3,11]

Você, com fome

Cliente — o programa que faz a chamada

Cardápio

Interface remota — o que pode ser pedido

Garçom

Stub — leva o pedido e traz a resposta

Comanda

Serialização — o pedido "anotado" para viajar

Cozinha

Objeto remoto — quem realmente faz o trabalho

Guia de restaurantes

Registro RMI — encontra o objeto pelo nome

E quando a cozinha fecha no meio do pedido? Aí aparece a `RemoteException`.[1]

[1] ORACLE. Java Remote Method Invocation Specification. Java SE 25. [3] ORACLE. An Overview of RMI Applications. The Java Tutorials. [11] COULOURIS, G. et al. Sistemas distribuídos: conceitos e projeto. 5. ed. Bookman, 2013.

UM POUCO DE HISTÓRIA

De onde veio o RMI

1984

RPC

Birrell e Nelson (Xerox PARC) mostram como chamar um procedimento em outra máquina como se fosse local.[12]

1994

O alerta

Waldo e colegas (Sun) mostram que esconder a rede por completo é um erro.[13]

1996

O modelo

Wollrath, Riggs e Waldo descrevem o modelo de objetos distribuídos do Java.[2]

1997

JDK 1.1

O RMI passa a fazer parte oficialmente do Java.[14]

Hoje

Java 25

Ainda presente no módulo `java.rmi` — mais enxuto e mais seguro.[15]

[2] WOLLRATH, A.; RIGGS, R.; WALDO, J. A distributed object model for the Java system. Computing Systems, v. 9, n. 4, 1996. **[12]** BIRRELL, A. D.; NELSON, B. J. Implementing remote procedure calls. ACM TOCS, v. 2, n. 1, 1984. **[13]** WALDO, J. et al. A note on distributed computing. Sun Microsystems Laboratories, TR-94-29, 1994. **[14]** SUN MICROSYSTEMS. Sun ships JDK 1.1: JavaBeans included. Comunicado de imprensa, 19 fev. 1997. **[15]** ORACLE. Module java.rmi. Java SE 25 & JDK 25 API Specification.

A LIÇÃO DE 1994

Remoto *não é* local

Quatro diferenças que nenhuma tecnologia consegue esconder por completo.[13]

01

Latência

Em 1994, uma chamada pela rede era de 10 mil a 100 mil vezes mais lenta que uma local.[13]

02

Memória

Um endereço de memória daqui não significa nada na outra máquina.[13]

03

Falha parcial

A rede ou o servidor podem cair no meio da chamada — e o cliente nem sabe se o pedido foi feito.[13]

04

Concorrência

Vários clientes podem chamar o mesmo objeto ao mesmo tempo.[13]

Por isso, no RMI, todo método remoto declara `throws RemoteException`: o Java **obriga** você a pensar na falha.[2,1]

[1] ORACLE. Java Remote Method Invocation Specification. Java SE 25.

[2] WOLLRATH, A.; RIGGS, R.; WALDO, J. A distributed object model for the Java system. Computing Systems, v. 9, n. 4, 1996.

[13] WALDO, J. et al. A note on distributed computing. Sun Microsystems Laboratories, TR-94-29, 1994.

02

APRESENTADOR 2

Conceitos básicos

As peças de uma aplicação RMI — e como elas **conversam** entre si.

[1] ORACLE. Java Remote Method Invocation Specification. Java SE 25.

VOCABULÁRIO

Os seis personagens

Cardápio

Interface remota

O contrato: os métodos que podem ser chamados de longe.[1]

Cozinha

Objeto remoto

Implementa a interface e fica no servidor esperando chamadas.[1]

Garçom

Stub

O representante do objeto remoto no cliente. Tem a mesma interface.[1]

Quem recebe na cozinha

Skeleton

O par do stub no servidor: recebe a chamada e aciona o objeto real.[1]

Comanda

Marshalling

Empacotar parâmetros e resultados em bytes, usando serialização.[1]

Guia de restaurantes

Registro

Um catálogo que liga um nome à referência do objeto.[16]

[1] ORACLE. Java Remote Method Invocation Specification. Java SE 25. [16] ORACLE. Getting Started Using Java RMI. Java SE 8 Documentation.

PEÇA 1 · O CARDÁPIO

A interface remota

HELLO.JAVA

```java
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface Hello extends Remote {
    String sayHello()
        throws RemoteException;
}
```

Exemplo do tutorial oficial da Oracle.[16]

1 **`extends Remote`**

Uma “etiqueta”: os métodos desta interface podem ser chamados de outra JVM.[1]

2 **`throws RemoteException`**

Exceção checada: a rede pode falhar — e quem chama é obrigado a tratar.[1,17]

3 **O que pode viajar**

Tipos primitivos, objetos `Serializable` ou outros objetos remotos.[3]

[1] ORACLE. Java Remote Method Invocation Specification. Java SE 25.
[3] ORACLE. An Overview of RMI Applications. The Java Tutorials.
[16] ORACLE. Getting Started Using Java RMI. Java SE 8 Documentation.
[17] ORACLE. The Catch or Specify Requirement. The Java Tutorials.

PEÇA 2 · A COZINHA

O objeto remoto

SERVER.JAVA

```java
public class Server implements Hello {

    public String sayHello() {
        return "Hello, world!";
    }

    public static void main(String[] args) throws Exception {
        Server obj = new Server();
        Hello stub = (Hello)
            UnicastRemoteObject.exportObject(obj, 0);
        // registrar o stub → parte 04
    }
}
```

1 Cumpre o contrato

A classe implementa a interface remota — é aqui que o trabalho de verdade acontece.[16]

2 “Abre a cozinha”

`exportObject` coloca o objeto no ar para receber chamadas. O `0` deixa o sistema escolher a porta.[3,18]

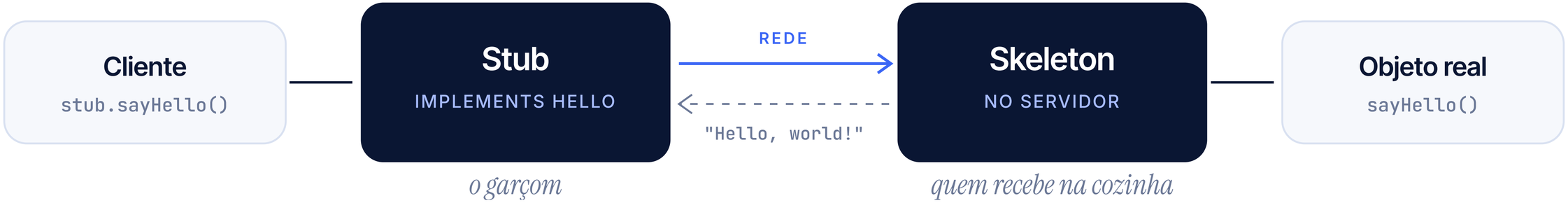
3 Devolve o stub

O “garçom” que depois será entregue aos clientes.[18]

[3] ORACLE. An Overview of RMI Applications. The Java Tutorials. [16] ORACLE. Getting Started Using Java RMI. Java SE 8 Documentation. [18] ORACLE. Class UnicastRemoteObject. Java SE 25 API.

PEÇA 3 · O GARÇOM

Stub e skeleton: os *intermediários*



Stub

Abre a conexão, empacota os parâmetros, espera e desempacota a resposta.[1]

Skeleton

Desempacota a chamada, aciona o objeto real e empacota o resultado.[1]

Padrão Proxy

O stub é um **proxy**: um representante que se passa pelo objeto verdadeiro.[19]

[1] ORACLE. Java Remote Method Invocation Specification. Java SE 25. [19] JGURU. Fundamentals of RMI: short course. Sun Microsystems, 2000.

COMO OS DADOS VIAJAM

Cópia *ou* referência?

OBJETO COMUM · SERIALIZABLE

Vai uma *cópia*

Como mandar uma **fotocópia**: rabiscar a cópia não muda o original.[3,1]

Campos `static` e `transient` não viajam.[3]

OBJETO REMOTO · REMOTE

Vai o *stub*

Como mandar o **número de telefone**: quem liga fala com o original.[3,1]

Ou seja: passagem “por referência” pela rede.[3]

Tipos primitivos (`int`, `double`...) vão por valor — como sempre foi no Java.[3]

[1] ORACLE. Java Remote Method Invocation Specification. Java SE 25.
[3] ORACLE. An Overview of RMI Applications. The Java Tutorials.

A COMANDA

Marshalling: empacotar para viajar



Empacotar

Parâmetros e resultados viram uma sequência de bytes.[1]

Com serialização

O RMI usa a serialização de objetos do próprio Java para isso.[1,9]

Curiosidade

Todo fluxo serializado começa com a "assinatura" `AC ED`.[8]

[1] ORACLE. Java Remote Method Invocation Specification. Java SE 25. [8] ORACLE. Java Object Serialization Specification. Java SE 25. [9] ORACLE. Object Streams. The Java Tutorials.

NOS BASTIDORES

Coleta de lixo... *distribuída*

O coletor de lixo do Java apaga objetos que ninguém usa. Mas como saber se um cliente **em outro computador** ainda usa o objeto?[1]

A SOLUÇÃO · ALUGUEL COM PRAZO (LEASE)

Como um **empréstimo de biblioteca**: o cliente pega o objeto com prazo e renova enquanto estiver usando. Se ele sumir, o prazo vence e o servidor pode liberar o objeto.[20]



Prazo padrão: **10 minutos** (`java.rmi.dgc.leaseValue`).[21]

[1] ORACLE. Java Remote Method Invocation Specification. Java SE 25. [20] ORACLE. Interface DGC. Java SE 25 API. [21] ORACLE. Java RMI Specification – Properties. Java SE 25.

JUNTANDO AS PEÇAS

A receita em *4 passos*

1

Definir a interface

Criar o “cardápio”: `extends Remote`.[3]

2

Implementar e exportar

Criar o objeto remoto e colocá-lo no ar.[16]

3

Registrar com um nome

Anunciar o objeto no `rmiregistry`.[16]

4

Procurar e chamar

O cliente faz `lookup` e usa o stub.[16]

Os passos 1 e 2 já vimos. O que acontece “por baixo” de cada chamada é o assunto da próxima parte.

[3] ORACLE. An Overview of RMI Applications. The Java Tutorials. [16] ORACLE. Getting Started Using Java RMI. Java SE 8 Documentation.

03

APRESENTADOR 3

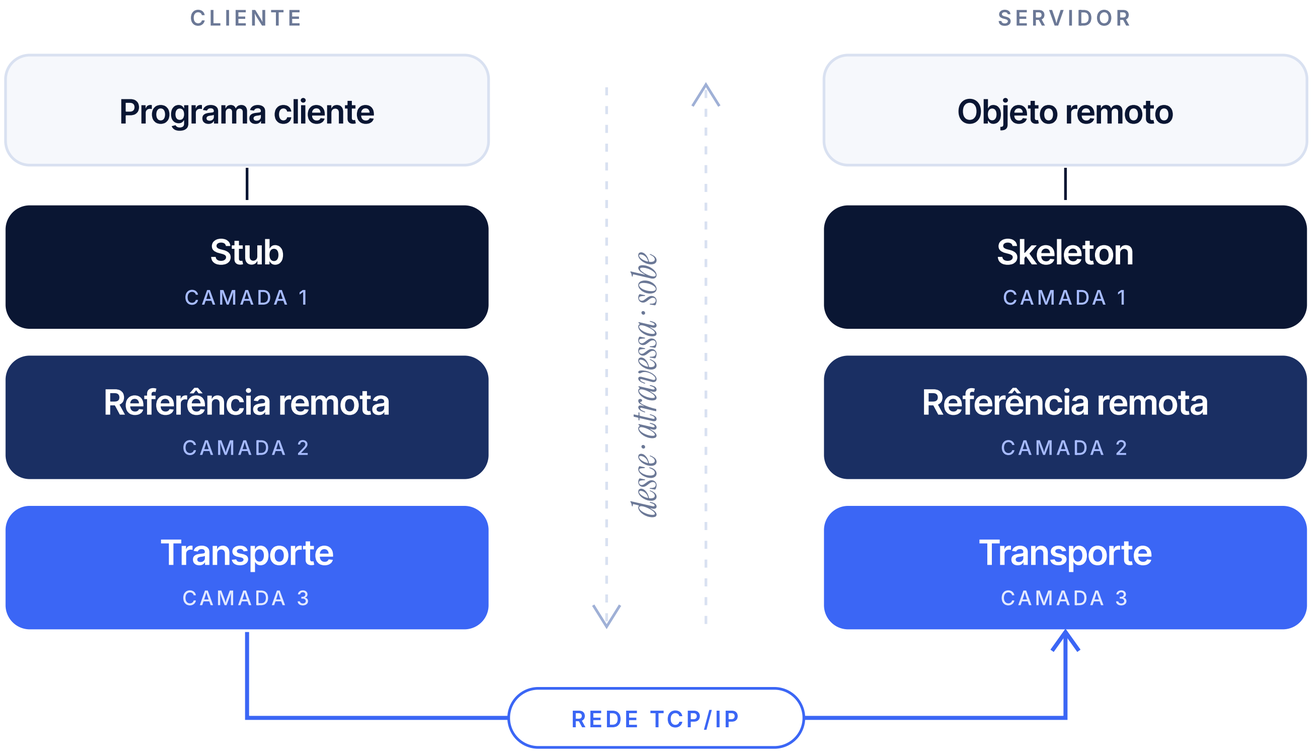
Camadas do RMI

O caminho de uma chamada remota, **andar por andar**.

[19] JGURU. Fundamentals of RMI: short course. Sun Microsystems, 2000. [22] SUN MICROSYSTEMS. Java Remote Method Invocation Specification, cap. 3: System Architecture. JDK 1.1, 1997.

VISÃO GERAL

Três camadas, dos dois lados



ANALOGIA · OS CORREIOS

1. **Envelope**
   Prepara e abre a mensagem.
2. **Endereço e tipo de envio**
   Para quem vai e como entregar.
3. **Caminhão e estrada**
   Leva a mensagem de fato.

Divisão descrita na especificação original de 1997: cada camada pode ser trocada sem afetar as outras. A especificação atual já não usa esse modelo, mas ele continua ótimo para aprender.[22,19,1]

[1] ORACLE. Java Remote Method Invocation Specification. Java SE 25.
[19] JGURU. Fundamentals of RMI: short course. Sun Microsystems, 2000.
[22] SUN MICROSYSTEMS. Java Remote Method Invocation Specification, cap. 3: System Architecture. JDK 1.1, 1997.

CAMADA 1

Stub e skeleton: a *porta de entrada*

É a fronteira entre o seu código e o sistema RMI: transforma uma chamada de método em mensagem — e a mensagem de volta em chamada.[22]

1997

JDK 1.1

Stub e skeleton gerados pela ferramenta `rmic`.[22]

1998

JDK 1.2

Novo protocolo: o skeleton deixa de ser necessário.[1,19]

2004

Java 5

Stubs criados automaticamente, na hora da execução.[23]

2020

JDK 15

A ferramenta `rmic` é removida do Java.[24]

Hoje: o stub é um `java.lang.reflect.Proxy` gerado em tempo de execução — ninguém precisa escrevê-lo nem gerá-lo à mão.[18]

[1] ORACLE. Java Remote Method Invocation Specification. Java SE 25. [18] ORACLE. Class UnicastRemoteObject. Java SE 25 API. [19] JGURU. Fundamentals of RMI: short course. Sun Microsystems, 2000. [22] SUN MICROSYSTEMS. Java Remote Method Invocation Specification, cap. 3: System Architecture. JDK 1.1, 1997. [23] ORACLE. Java RMI Release Notes. Java SE 8 Documentation. [24] OPENJDK. JDK-8225319: Remove rmic from the set of supported tools. JBS, 2020.

CAMADA 2

Referência remota: *para quem* e *como*

Define a “regra de entrega” da chamada — independentemente de como o stub foi feito.[22]

No código, ela aparece como um objeto `RemoteRef`.[19]

Unicast · padrão

Ponto a ponto

Um cliente conversa com um único objeto, que já está rodando.[19]

Ativação · 1998 → 2021

Objeto que “acorda” sob demanda

Criada no Java 2 e removida no Java 17 por falta de uso.[19,25]

Previsto no projeto

Objetos replicados

A camada foi pensada para permitir outras semânticas, como réplicas.[22]

[19] JGURU. Fundamentals of RMI: short course. Sun Microsystems, 2000. [22] SUN MICROSYSTEMS. Java Remote Method Invocation Specification, cap. 3: System Architecture. JDK 1.1, 1997. [25] OPENJDK. JEP 407: Remove RMI Activation. 2021.

CAMADA 3

Transporte: a *estrada*

Abre e gerencia as conexões, acompanha os objetos remotos e entrega cada chamada no destino certo.[22]

Conexão

TCP/IP

Conexões de rede confiáveis, baseadas em fluxo de bytes.[19]

Protocolo

JRMP

Java Remote Method Protocol: o "idioma" próprio do RMI na rede.[19,18]

O que mudou

Adeus ao "túnel" HTTP

Embrulhar chamadas em HTTP para passar por firewalls foi removido no JDK 9.[1]

[1] ORACLE. Java Remote Method Invocation Specification. Java SE 25. **[18]** ORACLE. Class UnicastRemoteObject. Java SE 25 API. **[19]** JGURU. Fundamentals of RMI: short course. Sun Microsystems, 2000. **[22]** SUN MICROSYSTEMS. Java Remote Method Invocation Specification, cap. 3: System Architecture. JDK 1.1, 1997.

JUNTANDO TUDO

Uma chamada, do começo ao fim

IDA · NO CLIENTE

01

O programa chama `stub.sayHello()`.

02

O stub empacota a chamada (marshalling).

03

A referência remota decide para onde e como enviar.

04

O transporte manda os bytes pela rede.

CHEGADA · NO SERVIDOR

05

O transporte recebe e entrega à camada de cima.

06

A chamada é desempacotada (unmarshalling).

07

O objeto real executa `sayHello()`.

08

A resposta faz o caminho inverso e chega como se fosse **local**.

Sequência baseada na descrição de stubs, skeletons e camadas das especificações do RMI.[22,1,11]

[1] ORACLE. Java Remote Method Invocation Specification. Java SE 25.
[11] COULOURIS, G. et al. Sistemas distribuídos: conceitos e projeto. 5. ed. Bookman, 2013.
[22] SUN MICROSYSTEMS. Java Remote Method Invocation Specification, cap. 3: System Architecture. JDK 1.1, 1997.

04

APRESENTADOR 3

Serviços de registro e naming

Como o cliente descobre **onde** está o objeto que ele quer usar?

[26] ORACLE. The rmiregistry Command. Java SE 25 & JDK 25.

O GUIA DE RESTAURANTES

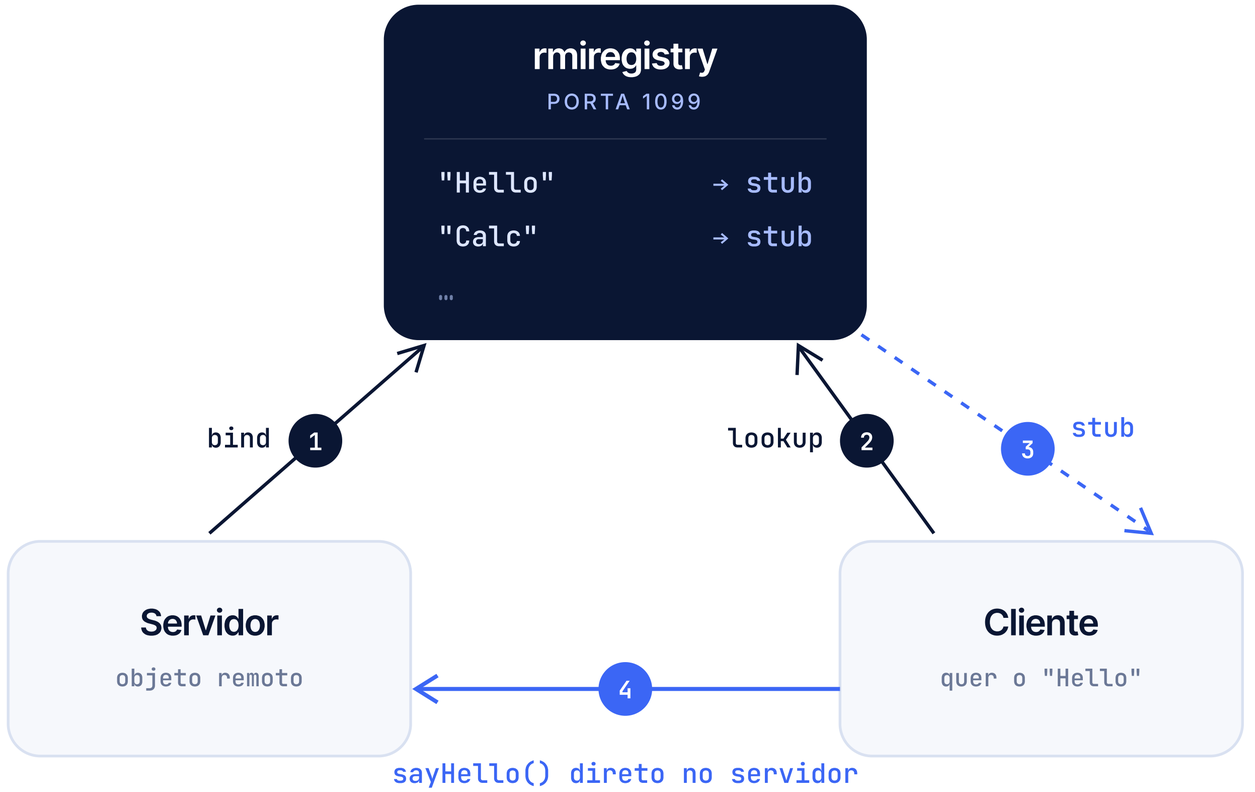
O *rmiregistry*

Um serviço de nomes simples: guarda pares **nome → stub** e entrega o stub a quem pedir.[16,26]

Porta padrão: 1099.[26]

Serve para achar o primeiro objeto — ele pode indicar os demais.[27]

Simples de propósito: lista plana e sem memória — se reiniciar, os nomes somem.[1]



[1] ORACLE. Java Remote Method Invocation Specification. Java SE 25. **[16]** ORACLE. Getting Started Using Java RMI. Java SE 8 Documentation. **[26]** ORACLE. The rmiregistry Command. Java SE 25 & JDK 25. **[27]** SUN MICROSYSTEMS. rmiregistry: the Java remote object registry. J2SE 5.0, 2004.

NA PRÁTICA

Registrar e procurar: *três linhas*

0 · TERMINAL

```
# sobe o registro
# na porta 1099
rmiregistry

# no Windows:
start rmiregistry
```

1 · SERVIDOR ANUNCIA

```
Registry registry =
  LocateRegistry
    .getRegistry();

registry.bind("Hello", stub);
```

2 · CLIENTE PROCURA E USA

```
Registry registry =
  LocateRegistry.getRegistry(host);

Hello stub = (Hello)
  registry.lookup("Hello");

stub.sayHello();
```

Trechos do tutorial oficial.[16,26]

Pegadinha: `getRegistry` não testa a conexão — o erro só aparece na primeira chamada.[28]

Atalho: `createRegistry(1099)` cria o registro dentro do próprio programa.[28]

[16] ORACLE. Getting Started Using Java RMI. Java SE 8 Documentation. [26] ORACLE. The rmiregistry Command. Java SE 25 & JDK 25. [28] ORACLE. Class LocateRegistry. Java SE 25 API.

TRÊS JEITOS DE CHEGAR AO MESMO LUGAR

Naming, Registry e JNDI

API	COMO FUNCIONA	EXEMPLO
java.rmi.Naming	Métodos estáticos que recebem um endereço no formato URL.[29]	`Naming.lookup("rmi://host:1099/Hello")`
Registry + LocateRegistry	Um objeto que representa o registro. É o estilo do tutorial oficial.[30,16]	`registry.lookup("Hello")`
JNDI	API genérica de nomes do Java: o mesmo código conversa com RMI, LDAP, DNS...[31]	`ctx.lookup("rmi://host/Hello")`

OPERAÇÕES **BIND** REBIND UNBIND **LOOKUP** LIST — bind/rebind/unbind só a partir da própria máquina.[32]

Comparando: o **DNS** é hierárquico e espalhado pelo mundo (`www.ufx.br`); o rmiregistry é uma **lista plana** em uma única máquina.[33,34]

[16] ORACLE. Getting Started Using Java RMI. Java SE 8 Documentation. **[29]** ORACLE. Class Naming. Java SE 25 API. **[30]** ORACLE. Interface Registry. Java SE 25 API. **[31]** ORACLE. RMI Registry Service Provider for the JNDI. Java SE 8 Documentation. **[32]** ORACLE. Implementing a Remote Interface. The Java Tutorials. **[33]** MOCKAPETRIS, P. RFC 1034: Domain names – concepts and facilities. IETF, 1987. **[34]** TANENBAUM, A. S.; STEEN, M. van. Sistemas distribuídos: princípios e paradigmas. 2. ed. Pearson, 2007.

05

APRESENTADOR 4

Aplicações de RMI

Onde ele foi — e ainda é — usado, e **o que mudou** de 1997 até 2026.

[15] ORACLE. Module java.rmi. Java SE 25 & JDK 25 API Specification.

NO MUNDO REAL

Onde o RMI aparece

Ainda hoje · JMX

Monitorar a JVM à distância

Ferramentas como o JConsole observam memória e threads de um servidor Java usando RMI.[35,36]

Anos 2000 · EJB

Sistemas corporativos

Componentes Enterprise JavaBeans seguiam o modelo do RMI, com interfaces remotas e `RemoteException`.[37]

1999 → 2022 · Jini

Serviços que se descobrem

O Jini (depois Apache River) usava o RMI como "língua" base. Foi aposentado em 2022.[38,39]

Pesquisa · clusters

Computação paralela

O Manta deixou a chamada RMI vazia 35× mais rápida que no JDK 1.2.[40]

Sala de aula

Ensino de sistemas distribuídos

Chat, calculadora remota, "compute engine": o laboratório clássico da disciplina.[41,11]

A grande herança

Uma ideia que ficou

Interface + stub + serialização: o mesmo trio vive hoje no gRPC.[42]

[11] COULOURIS, G. et al. Sistemas distribuídos: conceitos e projeto. 5. ed. Bookman, 2013. **[35]** ORACLE. Monitoring and Management Using JMX Technology. Java SE 25 Monitoring and Management Guide. **[36]** JAVA COMMUNITY PROCESS. JSR 160: Java Management Extensions (JMX) Remote API. 2003. **[37]** ORACLE. Enterprise JavaBeans Components and CORBA Clients: a developer guide. Java SE 8 Documentation. **[38]** SUN MICROSYSTEMS. Jini Architecture Specification, version 1.0. Apache River. **[39]** THE APACHE ATTIC. Apache River. 2022. **[40]** MAASSEN, J. et al. Efficient Java RMI for parallel programming. ACM TOPLAS, v. 23, n. 6, 2001. **[41]** ORACLE. Trail: RMI. The Java Tutorials. **[42]** GRPC AUTHORS. Introduction to gRPC. grpc.io.

MÃO NA MASSA

Calculadora remota em *3 arquivos*

1 · O CARDÁPIO

```java
// Calculadora.java
public interface Calculadora
    extends Remote {

  int somar(int a, int b)
    throws RemoteException;
}
```

2 · A COZINHA

```java
// Servidor.java
public class Servidor implements Calculadora {
  public int somar(int a, int b) { return a + b; }

  // referências estáticas: evitam a coleta de lixo
  static Servidor obj = new Servidor();
  static Registry reg;

  public static void main(String[] a) throws Exception {
    Calculadora stub = (Calculadora)
      UnicastRemoteObject.exportObject(obj, 0);
    reg = LocateRegistry.createRegistry(1099);
    reg.rebind("calc", stub);
    System.out.println("Servidor pronto!");
  }
}
```

3 · O CLIENTE

```java
// Cliente.java
Registry reg = LocateRegistry
    .getRegistry("localhost");

Calculadora calc = (Calculadora)
    reg.lookup("calc");

System.out.println(
    calc.somar(2, 3)); // 5
```

Para rodar: `javac *.java` → `java Servidor` em um terminal → `java Cliente` em outro.

Adaptado do tutorial oficial “Hello” (imports de `java.rmi.*` omitidos). `createRegistry` dispensa rodar o `rmiregistry` separado.[16,28]

[16] ORACLE. Getting Started Using Java RMI. Java SE 8 Documentation. [28] ORACLE. Class LocateRegistry. Java SE 25 API.

O CALCANHAR DE AQUILES

Segurança: cuidado com o que chega *pela rede*

Desserializar é **montar um objeto a partir de bytes recebidos**. Se um atacante preparar esses bytes, a montagem pode executar código dele.[43,44]

2015 — **O alerta.** A palestra “Marshalling Pickles” e a ferramenta ysoserial mostram ataques reais, inclusive contra o registro RMI.[43]

2017 — **A defesa.** Filtros de desserialização: registro e DGC passam a aceitar só uma lista de classes permitidas.[45,46]

2021 — **Log4Shell.** Nota máxima de gravidade (10.0); o governo americano recomendou bloquear RMI e LDAP de saída.[47,48]

2023 — **A pesquisa.** Estudo com 104 vulnerabilidades de desserialização em Java: 37,5% ainda sem correção.[44]

Regra prática: RMI só dentro de rede confiável, com filtros de desserialização ativos.[45,48]

[43] FROHOFF, C.; LAWRENCE, G. Marshalling pickles: how deserializing objects will ruin your day. AppSecCali, 2015. **[44]** SAYAR, I. et al. An in-depth study of Java deserialization remote-code execution exploits and vulnerabilities. ACM TOSEM, v. 32, n. 1, 2023. **[45]** RIGGS, R. JEP 290: Filter Incoming Serialization Data. OpenJDK, 2016. **[46]** ORACLE. JDK 8u121 Release Notes. 2017. **[47]** NIST. CVE-2021-44228 Detail. National Vulnerability Database, 2021. **[48]** CISA. AA21-356A: Mitigating Log4Shell and Other Log4j-Related Vulnerabilities. 2021.

PANORAMA · 1997 → 2026

O que mudou em *três décadas*

1997

JDK 1.1

O RMI nasce.[14]

1998

JDK 1.2

Adeus skeleton; chega a ativação.[1]

2004

Java 5

Stubs gerados sozinhos.[23]

2013

Java 7u21

Baixar código pela rede fica desligado por padrão.[23]

2017

JEP 290

Filtros de desserialização.[45]

2018

Java 11

Saem CORBA e RMI-IIOP.[49]

2020

JDK 15

A ferramenta `rmic` é removida.[24]

2021

Java 17

Ativação removida.[50,25]

2022

Spring 6

O framework abandona o suporte a RMI.[51]

2025

JDK 24 · 25

Security Manager desligado; `java.rmi` segue no JDK 25 LTS.[52,15]

[1] ORACLE. Java Remote Method Invocation Specification. Java SE 25. **[14]** SUN MICROSYSTEMS. Sun ships JDK 1.1: JavaBeans included. Comunicado de imprensa, 19 fev. 1997. **[15]** ORACLE. Module java.rmi. Java SE 25 & JDK 25 API Specification. **[23]** ORACLE. Java RMI Release Notes. Java SE 8 Documentation. **[24]** OPENJDK. JDK-8225319: Remove rmic from the set of supported tools. JBS, 2020. **[25]** OPENJDK. JEP 407: Remove RMI Activation. 2021. **[45]** RIGGS, R. JEP 290: Filter Incoming Serialization Data. OpenJDK, 2016. **[49]** ANDERSEN, L. JEP 320: Remove the Java EE and CORBA Modules. OpenJDK, 2017. **[50]** MARKS, S. JEP 385: Deprecate RMI Activation for Removal. OpenJDK, 2020. **[51]** SPRING. Deprecate remoting technologies support (#25379); RMI removido no 6.0 (#34779). GitHub. **[52]** MULLAN, S. JEP 486: Permanently Disable the Security Manager. OpenJDK, 2024.

PANORAMA

Antes × agora

	ANOS 1990–2000	2026
Papel	Principal jeito de fazer objetos distribuídos em Java.[2]	Nicho: monitoramento (JMX), sistemas antigos e ensino.[35,50]
Stubs	Gerados pelo `rmic`, junto com skeletons.[22]	Criados sozinhos em tempo de execução; `rmic` removido.[18,24]
Código pela rede	O cliente podia baixar classes de outro computador.[2]	Desligado desde 2013; impossível desde o JDK 24.[52]
Segurança	Aceitava qualquer objeto serializável.[45]	Filtros de desserialização e registro alterável só localmente.[45,32]
Vizinhos	CORBA, RMI-IIOP, EJB, Jini.[49,38]	gRPC, REST (HTTP + JSON), mensageria.[42,53]

[2] WOLLRATH, A.; RIGGS, R.; WALDO, J. A distributed object model for the Java system. Computing Systems, v. 9, n. 4, 1996. **[18]** ORACLE. Class UnicastRemoteObject. Java SE 25 API. **[22]** SUN MICROSYSTEMS. Java Remote Method Invocation Specification, cap. 3: System Architecture. JDK 1.1, 1997. **[24]** OPENJDK. JDK-8225319: Remove rmic from the set of supported tools. JBS, 2020. **[32]** ORACLE. Implementing a Remote Interface. The Java Tutorials. **[35]** ORACLE. Monitoring and Management Using JMX Technology. Java SE 25 Monitoring and Management Guide. **[38]** SUN MICROSYSTEMS. Jini Architecture Specification, version 1.0. Apache River. **[42]** GRPC AUTHORS. Introduction to gRPC. grpc.io. **[45]** RIGGS, R. JEP 290: Filter Incoming Serialization Data. OpenJDK, 2016. **[49]** ANDERSEN, L. JEP 320: Remove the Java EE and CORBA Modules. OpenJDK, 2017. **[50]** MARKS, S. JEP 385: Deprecate RMI Activation for Removal. OpenJDK, 2020. **[52]** MULLAN, S. JEP 486: Permanently Disable the Security Manager. OpenJDK, 2024. **[53]** FIELDING, R. T. Architectural styles and the design of network-based software architectures. Tese – UC Irvine, 2000.

A IDEIA SOBREVIVEU

Os *herdeiros* do RMI

1997 · Java

RMI

Só Java. A interface é uma `interface` Java; o stub é gerado na hora.[1]

2000 · Web

REST

Em vez de chamar métodos, manipula **recursos** via HTTP. Qualquer linguagem.[53]

2015 · várias linguagens

gRPC

Criado pelo Google. Interface num arquivo `.proto` e — adivinhe — um **stub** gerado automaticamente.[54,42]

A lição de 1994 continua valendo: mesmo nas tecnologias novas, uma chamada pela rede pode demorar, falhar ou precisar ser repetida — não dá para fingir que é local.[55,13]

[1] ORACLE. Java Remote Method Invocation Specification. Java SE 25. **[13]** WALDO, J. et al. A note on distributed computing. Sun Microsystems Laboratories, TR-94-29, 1994. **[42]** GRPC AUTHORS. Introduction to gRPC. grpc.io. **[53]** FIELDING, R. T. Architectural styles and the design of network-based software architectures. Tese – UC Irvine, 2000. **[54]** GRPC AUTHORS. About gRPC. grpc.io. **[55]** KLEPPMANN, M. Designing data-intensive applications. O'Reilly, 2017. Cap. 4.

PARA LEVAR PARA CASA

Cinco ideias

1 **RMI** = chamar métodos de um objeto em outra JVM como se ele estivesse aqui.[3]

2 A **interface remota** é o contrato: `extends Remote` + `RemoteException`.[1]

3 O **stub** é o representante; a **serialização** leva os dados.[1]

4 O **registro** é a lista telefônica: nome → stub.[16]

5 **Remoto nunca é local**: a rede demora, falha e precisa de segurança.[13]

[1] ORACLE. Java Remote Method Invocation Specification. Java SE 25. [3] ORACLE. An Overview of RMI Applications. The Java Tutorials. [13] WALDO, J. et al. A note on distributed computing. Sun Microsystems Laboratories, TR-94-29, 1994. [16] ORACLE. Getting Started Using Java RMI. Java SE 8 Documentation.

Obrigado.

Perguntas?

INTEGRANTES

Carlos Eduardo da Silva Pechiori

Gustavo Wendel Lopes da Silva

Rhuan Leonardo Baffero Marques

Sara Batista Pereira

PARA PENSAR

Se o RMI quer que a chamada remota pareça local, por que o Java *obriga* a tratar `RemoteException`?[2,13]

[2] WOLLRATH, A.; RIGGS, R.; WALDO, J. A distributed object model for the Java system. Computing Systems, v. 9, n. 4, 1996.

[13] WALDO, J. et al. A note on distributed computing. Sun Microsystems Laboratories, TR-94-29, 1994.

Referências

1 DE 3

[1] ORACLE. **Java Remote Method Invocation Specification**. Java SE 25. Redwood Shores: Oracle, 2025. Disponível em: https://docs.oracle.com/en/java/javase/25/docs/specs/rmi/index.html. Acesso em: 23 set. 2026. ATUAL · 2025

[2] WOLLRATH, A.; RIGGS, R.; WALDO, J. A distributed object model for the Java system. **Computing Systems**, Berkeley, v. 9, n. 4, p. 265-290, 1996. Versão original apresentada na 2nd USENIX Conference on Object-Oriented Technologies (COOTS), Toronto, 1996. Disponível em: https://www.usenix.org/legacy/publications/compsystems/1996/fall_wollrath.pdf. Acesso em: 23 set. 2026. CLÁSSICA · 1996

[3] ORACLE. **An overview of RMI applications**. In: THE JAVA TUTORIALS. Redwood Shores: Oracle, [2024]. Disponível em: https://docs.oracle.com/javase/tutorial/rmi/overview.html. Acesso em: 23 set. 2026. ATUAL · 2024

[4] ORACLE. **Lesson**: object-oriented programming concepts. In: THE JAVA TUTORIALS. Redwood Shores: Oracle, [2024]. Disponível em: https://docs.oracle.com/javase/tutorial/java/concepts/index.html. Acesso em: 23 set. 2026. ATUAL · 2024

[5] ORACLE. **What is an interface?** In: THE JAVA TUTORIALS. Redwood Shores: Oracle, [2024]. Disponível em: https://docs.oracle.com/javase/tutorial/java/concepts/interface.html. Acesso em: 23 set. 2026. ATUAL · 2024

[6] ORACLE. **Polymorphism**. In: THE JAVA TUTORIALS. Redwood Shores: Oracle, [2024]. Disponível em: https://docs.oracle.com/javase/tutorial/java/IandI/polymorphism.html. Acesso em: 23 set. 2026. ATUAL · 2024

[7] ORACLE. **Lesson**: exceptions. In: THE JAVA TUTORIALS. Redwood Shores: Oracle, [2024]. Disponível em: https://docs.oracle.com/javase/tutorial/essential/exceptions/index.html. Acesso em: 23 set. 2026. ATUAL · 2024

[8] ORACLE. **Java Object Serialization Specification**. Java SE 25. Redwood Shores: Oracle, 2025. Disponível em: https://docs.oracle.com/en/java/javase/25/docs/specs/serialization/index.html. Acesso em: 23 set. 2026. ATUAL · 2025

[9] ORACLE. **Object streams**. In: THE JAVA TUTORIALS. Redwood Shores: Oracle, [2024]. Disponível em: https://docs.oracle.com/javase/tutorial/essential/io/objectstreams.html. Acesso em: 23 set. 2026. ATUAL · 2024

[10] LINDHOLM, T.; YELLIN, F.; BRACHA, G.; BUCKLEY, A.; SMITH, D. **The Java Virtual Machine Specification**: Java SE 25 edition. Redwood Shores: Oracle, 2025. Disponível em: https://docs.oracle.com/javase/specs/jvms/se25/html/index.html. Acesso em: 23 set. 2026. ATUAL · 2025

[11] COULOURIS, G.; DOLLIMORE, J.; KINDBERG, T.; BLAIR, G. **Sistemas distribuídos**: conceitos e projeto. 5. ed. Porto Alegre: Bookman, 2013. Cap. 5: Invocação remota. ATUAL · 2013

[12] BIRRELL, A. D.; NELSON, B. J. Implementing remote procedure calls. **ACM Transactions on Computer Systems**, New York, v. 2, n. 1, p. 39-59, fev. 1984. DOI: 10.1145/2080.357392. Disponível em: https://dl.acm.org/doi/10.1145/2080.357392. Acesso em: 23 set. 2026. CLÁSSICA · 1984

[13] WALDO, J.; WYANT, G.; WOLLRATH, A.; KENDALL, S. **A note on distributed computing**. Mountain View: Sun Microsystems Laboratories, nov. 1994. (Technical Report SMLI TR-94-29). Disponível em: https://scholar.harvard.edu/files/waldo/files/waldo-94.pdf. Acesso em: 23 set. 2026. CLÁSSICA · 1994

[14] SUN MICROSYSTEMS. **Sun ships JDK 1.1**: JavaBeans included. Comunicado de imprensa. Palo Alto, 19 fev. 1997. Disponível em: https://web.archive.org/web/20080210044125/http://www.sun.com/smi/Press/sunflash/1997-02/sunflash.970219.0001.xml. Acesso em: 23 set. 2026. CLÁSSICA · 1997

[15] ORACLE. **Module java.rmi**. In: JAVA SE 25 & JDK 25 API Specification. Redwood Shores: Oracle, 2025. Disponível em: https://docs.oracle.com/en/java/javase/25/docs/api/java.rmi/module-summary.html. Acesso em: 23 set. 2026. ATUAL · 2025

[16] ORACLE. **Getting started using Java RMI**. Java SE 8 Documentation. Redwood Shores: Oracle, [2014]. Disponível em: https://docs.oracle.com/javase/8/docs/technotes/guides/rmi/hello/hello-world.html. Acesso em: 23 set. 2026. ATUAL · 2014

[17] ORACLE. **The catch or specify requirement**. In: THE JAVA TUTORIALS. Redwood Shores: Oracle, [2024]. Disponível em: https://docs.oracle.com/javase/tutorial/essential/exceptions/catchOrDeclare.html. Acesso em: 23 set. 2026. ATUAL · 2024

[18] ORACLE. **Class UnicastRemoteObject**. In: JAVA SE 25 & JDK 25 API Specification. Redwood Shores: Oracle, 2025. Disponível em: https://docs.oracle.com/en/java/javase/25/docs/api/java.rmi/java/rmi/server/UnicastRemoteObject.html. Acesso em: 23 set. 2026. ATUAL · 2025

[19] JGURU. **Fundamentals of RMI**: short course. [S. l.]: Sun Microsystems, 2000. Disponível em: https://teaching.shu.ac.uk/aces/pc/JDC/rmi/RMI.html. Acesso em: 23 set. 2026. CLÁSSICA · 2000

Referências no padrão ABNT NBR 6023:2018, numeradas na ordem da primeira citação. Clássica = até 2011 · Atual = 2012 em diante.

Referências

[20] ORACLE. **Interface DGC**. In: JAVA SE 25 & JDK 25 API Specification. Redwood Shores: Oracle, 2025. Disponível em: https://docs.oracle.com/en/java/javase/25/docs/api/java.rmi/java/rmi/dgc/DGC.html. Acesso em: 23 set. 2026. ATUAL · 2025

[21] ORACLE. **Java RMI specification**: properties. Java SE 25. Redwood Shores: Oracle, 2025. Disponível em: https://docs.oracle.com/en/java/javase/25/docs/specs/rmi/properties.html. Acesso em: 23 set. 2026. ATUAL · 2025

[22] SUN MICROSYSTEMS. **Java Remote Method Invocation Specification**: chapter 3 – system architecture. JDK 1.1. Mountain View: Sun Microsystems, 1997. Disponível em: https://www.cs.princeton.edu/courses/archive/fall97/cs461/jdkdocs/guide/rmi/spec/rmi-arch.doc.html. Acesso em: 23 set. 2026. CLÁSSICA · 1997

[23] ORACLE. **Java RMI release notes**. Java SE 8 Documentation. Redwood Shores: Oracle, [2014]. Disponível em: https://docs.oracle.com/javase/8/docs/technotes/guides/rmi/relnotes.html. Acesso em: 23 set. 2026. ATUAL · 2014

[24] OPENJDK. **JDK-8225319**: remove rmic from the set of supported tools. [S. l.]: Java Bug System, 2020. Disponível em: https://bugs.openjdk.org/browse/JDK-8225319. Acesso em: 23 set. 2026. ATUAL · 2020

[25] OPENJDK. **JEP 407**: remove RMI activation. [S. l.]: OpenJDK, 2021. Disponível em: https://openjdk.org/jeps/407. Acesso em: 23 set. 2026. ATUAL · 2021

[26] ORACLE. **The rmiregistry command**. Java SE 25 & JDK 25. Redwood Shores: Oracle, 2025. Disponível em: https://docs.oracle.com/en/java/javase/25/docs/specs/man/rmiregistry.html. Acesso em: 23 set. 2026. ATUAL · 2025

[27] SUN MICROSYSTEMS. **rmiregistry**: the Java remote object registry. J2SE 5.0 Documentation. [S. l.]: Sun Microsystems, 2004. Disponível em: https://docs.oracle.com/javase/1.5.0/docs/tooldocs/windows/rmiregistry.html. Acesso em: 23 set. 2026. CLÁSSICA · 2004

[28] ORACLE. **Class LocateRegistry**. In: JAVA SE 25 & JDK 25 API Specification. Redwood Shores: Oracle, 2025. Disponível em: https://docs.oracle.com/en/java/javase/25/docs/api/java.rmi/java/rmi/registry/LocateRegistry.html. Acesso em: 23 set. 2026. ATUAL · 2025

[29] ORACLE. **Class Naming**. In: JAVA SE 25 & JDK 25 API Specification. Redwood Shores: Oracle, 2025. Disponível em: https://docs.oracle.com/en/java/javase/25/docs/api/java.rmi/java/rmi/Naming.html. Acesso em: 23 set. 2026. ATUAL · 2025

[30] ORACLE. **Interface Registry**. In: JAVA SE 25 & JDK 25 API Specification. Redwood Shores: Oracle, 2025. Disponível em: https://docs.oracle.com/en/java/javase/25/docs/api/java.rmi/java/rmi/registry/Registry.html. Acesso em: 23 set. 2026. ATUAL · 2025

[31] ORACLE. **RMI registry service provider for the Java Naming and Directory Interface (JNDI)**. Java SE 8 Documentation. Redwood Shores: Oracle, [2014]. Disponível em: https://docs.oracle.com/javase/8/docs/technotes/guides/jndi/jndi-rmi.html. Acesso em: 23 set. 2026. ATUAL · 2014

[32] ORACLE. **Implementing a remote interface**. In: THE JAVA TUTORIALS. Redwood Shores: Oracle, [2024]. Disponível em: https://docs.oracle.com/javase/tutorial/rmi/implementing.html. Acesso em: 23 set. 2026. ATUAL · 2024

[33] MOCKAPETRIS, P. **RFC 1034**: domain names – concepts and facilities. [S. l.]: IETF, nov. 1987. Disponível em: https://www.rfc-editor.org/rfc/rfc1034. Acesso em: 23 set. 2026. CLÁSSICA · 1987

[34] TANENBAUM, A. S.; STEEN, M. van. **Sistemas distribuídos**: princípios e paradigmas. 2. ed. São Paulo: Pearson Prentice Hall, 2007. ISBN 978-85-7605-142-8. CLÁSSICA · 2007

[35] ORACLE. Monitoring and management using JMX technology. In: **Java SE 25 monitoring and management guide**. Redwood Shores: Oracle, 2025. Disponível em: https://docs.oracle.com/en/java/javase/25/management/monitoring-and-management-using-jmx-technology.html. Acesso em: 23 set. 2026. ATUAL · 2025

[36] JAVA COMMUNITY PROCESS. **JSR 160**: Java Management Extensions (JMX) remote API. [S. l.]: JCP, 2003. Disponível em: https://jcp.org/en/jsr/detail?id=160. Acesso em: 23 set. 2026. CLÁSSICA · 2003

[37] ORACLE. **Enterprise JavaBeans components and CORBA clients**: a developer guide. Java SE 8 Documentation. Redwood Shores: Oracle, [2014]. Disponível em: https://docs.oracle.com/javase/8/docs/technotes/guides/rmi-iiop/interop.html. Acesso em: 23 set. 2026. ATUAL · 2014

[38] SUN MICROSYSTEMS. **Jini architecture specification**: version 1.0. [S. l.]: Sun Microsystems, [1999]. Hospedado por Apache River. Disponível em: https://river.apache.org/release-doc/current/specs/html/jini-spec.html. Acesso em: 23 set. 2026. CLÁSSICA · 1999

Referências

[39] THE APACHE ATTIC. **Apache River**. [S. l.]: Apache Software Foundation, 2022. Disponível em: https://attic.apache.org/projects/river.html. Acesso em: 23 set. 2026. ATUAL · 2022

[40] MAASSEN, J.; NIEUWPOORT, R. V. van; VELDEMA, R.; BAL, H. E.; KIELMANN, T.; JACOBS, C. J. H.; HOFMAN, R. F. H. Efficient Java RMI for parallel programming. **ACM Transactions on Programming Languages and Systems**, New York, v. 23, n. 6, p. 747-775, nov. 2001. DOI: 10.1145/506315.506317. CLÁSSICA · 2001

[41] ORACLE. **Trail**: RMI. In: THE JAVA TUTORIALS. Redwood Shores: Oracle, [2024]. Disponível em: https://docs.oracle.com/javase/tutorial/rmi/index.html. Acesso em: 23 set. 2026. ATUAL · 2024

[42] GRPC AUTHORS. **Introduction to gRPC**. [S. l.]: The Linux Foundation, [2025]. Disponível em: https://grpc.io/docs/what-is-grpc/introduction/. Acesso em: 23 set. 2026. ATUAL · 2025

[43] FROHOFF, C.; LAWRENCE, G. **Marshalling pickles**: how deserializing objects will ruin your day. In: OWASP APPSECCALI, 2015, Santa Monica. Palestra. Disponível em: https://frohoff.github.io/appseccali-marshalling-pickles/. Acesso em: 23 set. 2026. ATUAL · 2015

[44] SAYAR, I.; BARTEL, A.; BODDEN, E.; LE TRAON, Y. An in-depth study of Java deserialization remote-code execution exploits and vulnerabilities. **ACM Transactions on Software Engineering and Methodology**, New York, v. 32, n. 1, art. 25, p. 1-45, fev. 2023. DOI: 10.1145/3554732. Disponível em: https://arxiv.org/abs/2208.08173. Acesso em: 23 set. 2026. ATUAL · 2023

[45] RIGGS, R. **JEP 290**: filter incoming serialization data. [S. l.]: OpenJDK, 2016. Disponível em: https://openjdk.org/jeps/290. Acesso em: 23 set. 2026. ATUAL · 2016

[46] ORACLE. **Java SE Development Kit 8, Update 121 (JDK 8u121)**: release notes. Redwood Shores: Oracle, 17 jan. 2017. Disponível em: https://www.oracle.com/java/technologies/javase/8u121-relnotes.html. Acesso em: 23 set. 2026. ATUAL · 2017

[47] NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY (NIST). **CVE-2021-44228 detail**. National Vulnerability Database. Gaithersburg: NIST, 10 dez. 2021. Disponível em: https://nvd.nist.gov/vuln/detail/CVE-2021-44228. Acesso em: 23 set. 2026. ATUAL · 2021

[48] CYBERSECURITY AND INFRASTRUCTURE SECURITY AGENCY (CISA). **AA21-356A**: mitigating Log4Shell and other Log4j-related vulnerabilities. Washington, DC: CISA, 22 dez. 2021. Disponível em: https://www.cisa.gov/news-events/cybersecurity-advisories/aa21-356a. Acesso em: 23 set. 2026. ATUAL · 2021

[49] ANDERSEN, L. **JEP 320**: remove the Java EE and CORBA modules. [S. l.]: OpenJDK, 2017. Disponível em: https://openjdk.org/jeps/320. Acesso em: 23 set. 2026. ATUAL · 2017

[50] MARKS, S. **JEP 385**: deprecate RMI activation for removal. [S. l.]: OpenJDK, 2020. Disponível em: https://openjdk.org/jeps/385. Acesso em: 23 set. 2026. ATUAL · 2020

[51] SPRING PROJECTS. **Deprecate remoting technologies support**: issue #25379. GitHub, 2020. Disponível em: https://github.com/spring-projects/spring-framework/issues/25379. Ver também: issue #34779 (remoção do RMI no Spring Framework 6.0). Disponível em: https://github.com/spring-projects/spring-framework/issues/34779. Acesso em: 23 set. 2026. ATUAL · 2020

[52] MULLAN, S. **JEP 486**: permanently disable the Security Manager. [S. l.]: OpenJDK, 2024. Disponível em: https://openjdk.org/jeps/486. Acesso em: 23 set. 2026. ATUAL · 2024

[53] FIELDING, R. T. **Architectural styles and the design of network-based software architectures**. 2000. Tese (Doutorado em Information and Computer Science) – University of California, Irvine, 2000. Disponível em: https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm. Acesso em: 23 set. 2026. CLÁSSICA · 2000

[54] GRPC AUTHORS. **About gRPC**. [S. l.]: The Linux Foundation, [2025]. Disponível em: https://grpc.io/about/. Acesso em: 23 set. 2026. ATUAL · 2025

[55] KLEPPMANN, M. **Designing data-intensive applications**: the big ideas behind reliable, scalable, and maintainable systems. Sebastopol: O'Reilly Media, 2017. Cap. 4: Encoding and evolution. ATUAL · 2017

Referências no padrão ABNT NBR 6023:2018, numeradas na ordem da primeira citação. Clássica = até 2011 · Atual = 2012 em diante.